

✓1) Exemplo - redirecionamento: Experimentar a seguinte sequência de comandos

Use ↑ para não ter de reescrever todo o comando.

✓\$ls -l	# lista ficheiros
✓\$ls -l tail -n +2	# salta 1a linha
✓\$ls -l tail -n +2 awk '{print \$9;}'	# “corta” a 9º col.
✓\$ls -l tail -n +2 awk '{print \$9;}' sort	# ordena
✓\$ls -l tail -n +2 awk '{print \$9;}' sort > files.txt	# redireciona para um ficheiro

✓2) Exemplo - sistema de ficheiros: Experimentar a seguinte sequência de comandos

✓\$cd	# ou cd ~
✓\$mkdir exemplo	
✓\$cd exemplo	
✓\$echo 'Bom dia Mundo' > ficheiro.txt	
✓\$cat ficheiro.txt	
✓\$echo 'Adeus' >> ficheiro.txt	
✓\$less ficheiro.txt	
✓\$mkdir subdir1/subdir2	✓# Erro, porquê?
✓\$mkdir -p subdir1/subdir2	
✓\$mv ficheiro.txt subdir1/subdir2	
✓\$cd ..	
✓\$rmdir exemplo	✓# Erro, porquê?
✓\$rm -rf exemplo	

✓3) Exercício

- ✓1. Criar as pastas ~/fcad/ex1/ e ~/fcad/ex1/subdir1
- ✓2. Criar o ficheiro f1.txt com o conteúdo 'fcad <nome do utilizador> <data>'
- ✓3. Mova o ficheiro para fcad/ex1
- ✓4. Repita o comando anterior “\$!!”, o que aconteceu?
- ✓5. Criar o ficheiro f2.txt com lista de ficheiros na pasta fcad
- ✓6. Criar o ficheiro f3.txt com a lista de ficheiros terminados em “.log” existentes na pasta /var/
- ✓7. Visualize o conteúdo dos ficheiros /proc/cpuinfo e /proc/meminfo. Qual a informação que contém?

Ficha 1

1. \$ ls -l

ls ⇒ Comando para listar arquivos e diretórios.

-l ⇒ Opção/Switch que ordenará ao comando ls uma listagem detalhada.

\$ ls -l | tail -n +2

| ⇒ Operador de pipe

tail -n +2 ⇒ Apresenta as últimas linhas do output de ls -l começando na linha 2.

\$ ls -l | tail -n +2 | awk '{print \$9;}'

awk '{print \$9;}' ⇒ Comando instruído a imprimir apenas o novo campo/coluna de cada linha.

\$ ls -l | tail -n +2 | awk '{print \$9;}' | sort

sort ⇒ Classifica a lista de arquivos e diretórios por ordem alfabética.

\$ ls -l | tail -n +2 | awk '{print \$9;}' | sort > file.txt

> ⇒ Redireciona o output do comando 'sort' para o arquivo 'file.txt'.

\$ cat file.txt

cat file.txt ⇒ Imprime o conteúdo do arquivo 'file.txt' no terminal.

> ⇒ Cria um novo arquivo ou substitui o conteúdo de um arquivo existente.

>> ⇒ Acrescenta conteúdo a um arquivo existente sem excluir o conteúdo anterior.

2. \$ cd ou \$ cd ~

Muda o diretório atual para o diretório 'home'.

\$ mkdir exemplo

Cria um novo diretório com o nome 'exemplo'.

\$ cd exemplo

Entra no diretório 'exemplo'.

\$ echo 'Bom dia Mundo' > ficheiro.txt

Cria um novo arquivo com o nome 'ficheiro.txt' e com o conteúdo 'Bom dia Mundo'.

\$ cat ficheiro.txt

Apresenta o conteúdo do arquivo 'ficheiro.txt' na linha de comandos.

\$ echo 'Adeus' >> ficheiro.txt

Acrescenta o conteúdo 'Adeus' ao final do arquivo 'ficheiro.txt' sem eliminar o conteúdo guardado anteriormente nele.

\$ less ficheiro.txt

Abre o arquivo 'ficheiro.txt' no visualizador de texto 'less'.

\$ mkdir subdir1/subdir2

Este comando resultará em erro pois tentamos criar um diretório 'subdir2' dentro de um diretório 'subdir1' que ainda não existe.

\$ mkdir -p subdir1/subdir2

Cria recursivamente o diretório 'subdir2' dentro do diretório 'subdir1', ou seja, cria todos os diretórios necessários até 'subdir2'.

\$ mv ficheiro.txt subdir1/subdir2

Move o arquivo 'ficheiro.txt' para o diretório 'subdir2' que se encontra dentro do diretório 'subdir1'.

\$ cd ..

Retorna ao diretório anterior, ao diretório pai.

\$ rmdir exemplo

Este comando tenta remover o diretório 'exemplo' sem êxito pois o diretório 'subdir2' ainda contém um arquivo.

O comando 'rmdir' apenas remove diretórios vazios.

\$ rm -rf exemplo

Remove recursivamente o diretório 'exemplo' e todo o conteúdo dentro dele, incluindo o arquivo 'ficheiro.txt' dentro do diretório 'subdir1'.

O parâmetro '-rf' indica recursivamente (remove todos os arquivos e diretórios) e forçadamente (remove o diretório e o seu conteúdo sem pedir confirmação).

Ficha 2 – Processos e Permissões

- Nesta ficha pode queres utilizar os comandos:
 - mkdir, cd, cp, ls, find
 - ctrl+c, ctrl+z, &, ps, kill
 - chmod, fg, bg, top, killall
- Descarregue os ficheiro loop.c e countdown do moodle.
- loop.c é um programa em c que fica num loop infinito
- countdown é um script de consola que faz uma contagem decrescente

Utilize a consola para fazer o seguinte

- ✓1) **Crie uma pasta, de onde irá realizar o trabalho.**
- ✓2) **Copie os ficheiros que acabou de descarregar.**
- ✓3) **Compile o programa loop.c**

```
$gcc -o loop loop.c
```

- ✓a. **Execute o programa loop.**
- ✓b. **Termine o programa**
- ✓c. **Execute o programa loop em background**
- ✓d. **Obtenha o PID do programa e termine-o com o sinal 15**

- ✓4) **Execute o script countdown com o argumento 10**
\$./countdown 10

Qual o erro obtido e porquê?

- ✓a. **Modifique as permissões do ficheiro countdown e execute novamente**
- ✓b. **Suspenda e retome a execução em foreground**
- ✓c. **Suspenda e retome a execução em background**
- ✓d. **Termine o processo.**

- ✓5) **Inicie 5 processos loop em background**
 - ✓a. **Verifique que estão todos em execução**
 - ✓b. **Termine todos**

Ficha 3 – Introdução às Variáveis e Scripts

Utilize a consola para fazer os seguintes exercícios.

1) Variáveis locais vs. variáveis globais

Vamos utilizar os comandos: *echo*, *ls*, *bash* e *ps* para verificar como as variáveis locais e globais podem (ou não) ser acedidas em consolas derivadas.

Execute a sequência de comandos na consola e responda às questões colocadas.

- ✓a. `$ myvar=Olá; myvar2="Bom dia."` Atribui valores às variáveis localmente.
- ✓b. `$ export myvar` Exporta a variável do ambiente local para o ambiente global.
- ✓c. `$ echo $myvar; echo $myvar2` Imprime ambas as variáveis.
- ✓d. `$ bash` Entra na *bash*.
- ✓e. `$ echo $myvar; echo $myvar2` Apenas imprime a variável 'myvar' pois apenas esta está definida no ambiente global.

1.1) Explique o que aconteceu? O que fará o comando `export`?

O comando 'export' vai exportar a variável local *myvar* para um ambiente global, permitindo aceder a esta variável noutra shell/linha de comandos.

- ✓f. `$ myvar=Adeus; myvar2="Vou embora."` Atribui valores às variáveis na *bash*.
- ✓g. `$ echo $myvar; echo $myvar2` Imprime ambas as variáveis.
- ✓h. `$ unset myvar; unset myvar2` Indefine ambas as variáveis.
- ✓i. `$ echo $myvar; echo $myvar2` Nenhuma das variáveis é impressa pois ambas foram indefinidas.

1.2) O que faz o comando `unset`?

O comando 'unset' remove a variável, indefinindo-a.

- ✓j. `$ exit` Sai da *bash*.
- ✓k. `$ echo $myvar; echo $myvar2` Voltámos à shell original por isso as variáveis voltaram a ser impressas, visto que estas tinham sido definidas localmente nesta mesma shell.

1.3) Justifique o resultado obtido com os comandos `echo` da linha anterior? O que fez o comando `exit`?

O comando 'exit' saiu da *bash* e entrou na shell anterior, por isso reproduziu as variáveis anteriormente definidas localmente nessa mesma shell.

Ficha 3 – Introdução às Variáveis e Scripts

✓2) Listas de valores.

Vamos explorar os vetores da consola. Execute os seguintes comandos na linha de comandos.

- ✓a. `$ my_array=(zero, um, dois, tres)` Atribui valores ao vetor 'my_array'.
- ✓b. `$ echo $my_array;` Imprime o primeiro valor atribuído a 'my_array' (zero).
- ✓c. `$ echo ${my_array[1]}` Imprime o valor atribuído a 'my_array' na posição [1] (um).
- ✓d. `$ echo ${my_array[*]}` Imprime todos os valores atribuídos ao vetor 'my_array'.

✓2.1 Os {} são necessários? Para que serve o [*]?

Os '{}' são necessários para que os '[...]' fiquem anexados a '\$my_array', caso contrário os comandos serão executados em separado. O '[*]' serve para imprimir não apenas um valor mas sim todos os valores associados àquele vetor/array.

- ✓e. `$ unset my_array[2]` Indefine o valor atribuído ao vetor na posição [2].
- ✓f. `$ echo ${my_array[*]}` Imprime todos os valores do vetor exceto o valor da posição [2] pois agora é indefinido.

✓2.2 O que faz o comando unset neste caso?

Neste caso o comando 'unset' vai definir apenas a variável anexada na posição [2], mantendo as restantes posições deste vetor inalteradas.

- ✓g. `$ echo ${my_array[2]}` Não é impresso qualquer valor.

✓2.3 Explique o que aconteceu?

'my_array[2]' é agora uma string vazia pois foi indefinida anteriormente, por isso não é impresso qualquer valor.

Ficha 3 – Introdução às Variáveis e Scripts

- ✓3) Indique a lista de comandos para mostrar a data, a pasta atual, e o nome do utilizador e execute-a na consola. Ex. (cmd1; cmd2; cmd3)

(date; pwd; whoami)

- ✓4) Crie uma pasta chamada lab3, e nessa pasta use o vim para criar um ficheiro chamado l3_file1, com os comandos da problema anterior.

```
$ mkdir lab3
$ vim l3_file1
#!/bin/bash
date
pwd
whoami
```

```
$ cat ~/lab3/l3_file1
date
pwd
whoami
```

- ✓4.1 Indique o comando para modificar as permissões do ficheiro de forma a este poder ser executado.

'\$ chmod +x l3_file1'

- ✓4.2 Modifique as permissões do ficheiro l3_ex1 e execute-o (\$ ~/lab3/l3_file1). O que aconteceu?

Ao executar o ficheiro, os comandos inseridos nele (date, pwd, whoami) foram executados e visualizados na shell.

- ✓4.3 Tente novamente com o comando source, e.g. \$ source ~/lab3/l3_file1. O que aconteceu agora? Verifique no manual (\$ man source) o que faz o comando source.

O comando 'source file1' é utilizado para executar os comandos de um ficheiro na pasta atual, equivalentemente a executar './file1'.

- ✓4.4 Utilizando novamente o vim, adicione a seguinte primeira linha

#!/bin/bash

ao ficheiro e tente executar novamente (\$ ~/lab3/l3_file1).

Funcionou? Porquê?

Funcionou pois nas novas shells executa de qualquer modo, anteriormente apenas executava com o comando 'source'.

Ficha 3 – Introdução às Variáveis e Scripts

✓4.5 Execute novamente o script, mas sem indicar o caminho (\$ 13_file1), o que aconteceu? Porquê?

○ comando não executou pois o caminho não estava completo, atirado.

✓4.6 A variável de ambiente PATH indica à Shell onde pode encontrar comandos externos, eliminando desta forma a necessidade de indicar o caminho completo de cada vez que se pretende executar um comando. Modifique a PATH para incluir a pasta lab3 (\$ PATH=\$PATH:~/lab3) e execute novamente o script (\$ 13_file1). O que aconteceu? Explique o que fez o comando \$ PATH=\$PATH:~/lab3.

Agora o comando executou, pois o comando '\$ PATH=\$PATH:~/lab3' definiu o caminho dos ficheiros que iria tentar executar.

✓4.7 Modifique o script de forma ao ser output passar a ser:

\$ 13_file1

A data é <DATA>

A pasta atual é <PWD>

O utilizador que está a executar este comando é <USER>

Indique o resultado.

```
$ vim 13_file1
#!/bin/bash
echo "A data é $(date)"
echo -n "A pasta atual é "
pwd
echo -n "O utilizador que está a executar este comando é "
```

✓5) Descarregue o ficheiro sampled.csv do moodle. CSV é um formato de texto para dados tabelados em que os campos estão separados por „

✓5.1 Com o comando wc conte as linhas do ficheiro. \$ wc -l sampled.csv

✓5.2 Com os comandos head e tail, respetivamente, mostre as primeiras 3 e últimas 5 linhas do ficheiro. \$ head -n 3 sampled.csv
\$ tail -n 5 sampled.csv

✓5.3 Com o comando sed crie uma cópia do ficheiro com o separador , substituído por /t (tab) \$ sed "s/,/\t/g" sampled.csv > sampled2.csv

✓5.4 Repita a operação anterior mas com o comando tr \$ tr ",","t" < sampled.csv > sampled3.csv

✓5.5 Com o comando cut, obtenha um ficheiro csv, apenas com as colunas w4 e l4. O separador por omissão do cut é o tab, mas pode ser indicado outro. \$ cut -d "," -f 4,7 sampled.csv

Ficha 4 – If e loops

Utilize a consola para fazer os seguintes exercícios.

1) Descarregue os dados para a realização deste laboratório

Descarregar o arquivo *dados.tgz* e utilize o comando *tar* para extrair o seu conteúdo.

1.1) Indique o comando (com todas as opções necessárias) que devem ser tar utilizadas.

```
$ tar -xzf dados.tgz
-x → Extrai os arquivos do arquivo tar.
-z → Usa gzip para descomprimir o arquivo.
-v → Envia detalhes sobre o processo de extração.
-f → Especifica o nome do arquivo a ser extraído(dados.tgz).
```

1.2) Qual é o conteúdo do ficheiro. Que comandos pode utilizar para o explorar?

Este arquivo possui pastas e arquivos.
Podemos explorar o ficheiro com recurso aos comandos *ls* e *cd*.

2) Organizar os ficheiros na pasta *data1* em 3 pastas, *log*, *txt*, *dat*, cada uma com todos os ficheiros *.log*, *.txt* e *.dat* respectivamente.

2.1 Qual/ Quais os comandos para criar as pastas *log*, *txt* e *dat*?

```
$ mkdir data1/log data1/txt data1/dat
```

2.2 Indique como mover todos os ficheiros terminados em *.log*, *.txt* e *.dat* para as as pastas *log*, *txt* e *dat*.

```
$ mv data1/*.log data1/log/
$ mv data1/*.txt data1/txt/
$ mv data1/*.dat data1/dat/
```

2.3 Indique os comandos a introduzir na consola para contar o número de ficheiros em cada pasta e produzir o resultado seguinte?

```
Number of files in log folder: 542
Number of files in txt folder: 551
Number of files in dat folder: 578
```

```
$ vim file_counter.dat
#!/bin/bash
echo "Number of files in log folder: $(ls log | wc -l)"
echo "Number of files in txt folder: $(ls txt | wc -l)"
echo "Number of files in dat folder: $(ls dat | wc -l)"
Clicar 'Esc' e digitar ':wq' no terminal do vim para guardar e sair.
```


Ficha 4 – If e loops

3) Análise do conteúdo da pasta data2

3.1 Crie um script de bash que:

- Conta e mostra o número de pastas dentro da pasta data2
- Conta e mostra o número de ficheiros em cada pasta
- Conta e mostra o número total de ficheiros na pasta data2
- Cria um ficheiro com o nome de todos os ficheiros no formato
folder_name/file_name

```
$ vim script_data2
#!/bin/bash
echo "Number of folders in 'data2': $(ls -l | grep "^d" | wc -l)"
echo "> filestat"
for folder in *
do
    if [ -d $folder ]; then
        echo "Number of files in $folder: $(ls -l $folder | grep "^-" | wc -l)"
        for file in $folder/*
        do
            echo $file >> files-list
        done
    fi
done
echo "Number of files in 'data2': $(ls -lR | grep "^-" | wc -l)"
```

Ficha 4 – If e loops

4) Análise do conteúdo da pasta data3

- ✓4.1 Crie um script de bash que escreve o nome de todos os ficheiros numa pasta com um ciclo.
- ✓4.2 Modifique o script anterior com a ajuda do comando wordcount (**wc**) para calcular o tamanho do máximo e mínimo dos ficheiros na pasta data3.
- 4.3 Com a ajuda do comando *stat*, indique um script de bash que escreve o nome de todos os ficheiros nessa pasta cujo tamanho seja superior a 990 bytes.
- 4.4 Crie um script de bash que copia todos os ficheiros com pelo menos um 7 no nome para a pasta *sete*.

```
4.1. $ vim script_4.1
#!/bin/bash
echo "> files-list-4.1"
for file in *.txt;
do
    if [ -f $file ]; then
        echo $file >> files-list-4.1
    fi
done
```

```
4.2. $ vim script_4.2
#!/bin/bash
echo "> files-list-4.2"
max=0
min=0
for file in *.txt;
do
    if [ -f $file ]; then
        tamanho=$(wc -c < $file)
        echo -e "$file\t$tamanho" >> files-list-4.2
        if [ $tamanho -gt $max ]; then
            max=$tamanho
        fi
        if [ $min -eq 0 ] || [ $tamanho -lt $min ]; then
            min=$tamanho
        fi
    fi
done
echo "Tamanho máximo = $max bytes"
echo "Tamanho mínimo = $min bytes"
```

```
4.3. $ vim script_4.3
#!/bin/bash
echo "> files-list-4.3"
for file in *.txt;
do
    if [ -f $file ] && [ $(stat -c %s $file) -gt 990 ]; then
        echo -e "$file\t$(wc -c < $file)" >> files-list-4.3
    fi
done
```

```
4.4. $ vim script_4.4
#!/bin/bash
for file in *.txt;
do
    if [[ $file == *7* ]]; then
        mkdir -p sete
        cp $file sete/
    fi
done
```


